

Cyber GYM: A Cloud-Native Cyber Range with Real-Time, AI-Assisted Log Analysis for Interactive Cybersecurity Education

Aditya Raj Pandit
23BRS1157

[pandit.aditya2023@vitstudent.
ac.in](mailto:pandit.aditya2023@vitstudent.ac.in)

Shivam Gupta
23BRS1167

[shivam.gupta2023@vitstudent.
ac.in](mailto:shivam.gupta2023@vitstudent.ac.in)

Amrit Raj
23BRS1103

amrit.raj2023@vitstudent.ac.in

Shreyash Khare
23BRS1039

[shreyash.khare2023@vitstudent.
ac.in](mailto:shreyash.khare2023@vitstudent.ac.in)

Prasun Singh
23BRS1117

[prasun.singh2023@vitstudent.
ac.in](mailto:prasun.singh2023@vitstudent.ac.in)

Abstract

The persistent gap between theoretical cybersecurity knowledge and practical, hands-on skills remains a major challenge in modern education, as traditional classrooms rarely provide safe, scalable, and realistic environments for simulating real cyber attacks. To address this limitation, this paper presents Cyber GYM, an interactive, cloud-native cyber range designed for controlled attack-defense training. Built on a secure multi-tier architecture using Amazon Web Services (AWS), the platform leverages EC2 for computation, RDS for persistent data storage, and a Virtual Private Cloud (VPC) for strong network isolation. Cyber GYM provides its core functionality through a browser-based interface that features a live terminal powered by WebSocket for real-time, full-duplex communication and secure SSH connectivity (via Paramiko) for remote execution on target virtual machines. The primary innovation of this work is the integration of a generative AI model, Google Gemini, as a real-time pedagogical assistant capable of analyzing user commands and system logs on the fly to deliver immediate explanations, threat classifications, and defensive recommendations. Alongside this, a machine learning module detects specific threat behaviors, such as the Mirai botnet. Cyber GYM ultimately offers a scalable, cost-effective, and secure solution for bridging the gap between cybersecurity theory and practical incident response.

Introduction

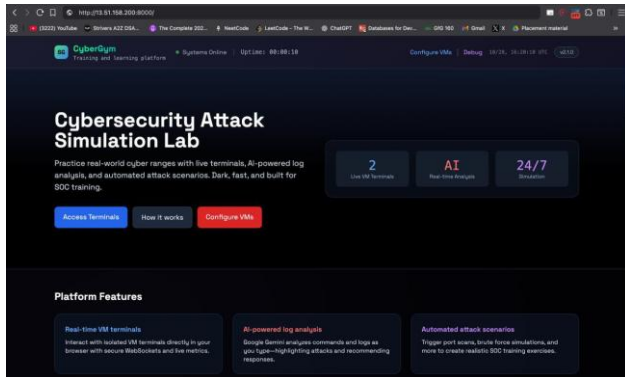
Cybersecurity has become one of the most critical domains in today's digital world. With the increasing number of cyber attacks, malware outbreaks, and large-scale botnet operations, traditional classroom-based learning is no longer sufficient to train students or professionals effectively. There is a need for practical, hands-on cybersecurity training environments where users can simulate real-world attack and defense operations.

Cyber GYM is a cloud-based cyber range platform designed to provide an interactive and safe environment for ethical hacking and defensive security practice. It allows users to execute attack simulations, monitor system performance, detect botnet behavior, and generate AI-based threat reports in real time. The system uses WebSockets for live terminal communication, SSH for remote VM access, and a structured microservices architecture powered by FastAPI. It also includes AI-driven threat intelligence analysis using Google Gemini and machine learning-based Mirai botnet detection.

Cyber GYM demonstrates how modern cybersecurity labs can be built using scalable cloud services, advanced backend technologies, and AI integration. It is a practical platform that bridges the gap between cybersecurity theory and real incident response learning through hands-on experimentation.

I. LITREATURE REVIEW

The development of the Cyber GYM platform is situated at the intersection of several active research fields: cloud-based cyber ranges, interactive training platforms, and the application of Artificial Intelligence to both cybersecurity operations and education. A review of the literature is necessary to position the novel contributions of this work.



A. Cloud-Based Cyber Ranges (CRaaS)

The concept of the cyber range has evolved from costly, static physical testbeds to highly flexible, virtualized environments. The "as-a-service" model has been applied to cyber ranges, leading to the "Cyber Range as a Service" (CRaaS) paradigm, which leverages cloud computing for scalable and on-demand training. Research in this area, such as the development of testbed frameworks for simulating attacks in IaaS (Infrastructure as a Service) clouds, highlights the benefits of using cloud platforms for security scenarios. These environments provide a realistic yet controlled sandbox where attacks can be studied without affecting real infrastructures. II. Author name(s) and affiliation(s)

B. Interactive Cybersecurity Training Platforms

The Cyber GYM methodology represents a fundamental pedagogical shift. It re-positions the AI from a curriculum designer to a *real-time, interactive pedagogical mentor*. The student is not in a "text-based simulation"; they are in a real Linux terminal, connected to a real virtual machine, executing real commands. The AI (Google Gemini) functions as an "over-the-shoulder" expert, observing the student's actions and providing immediate, context-aware feedback. This *in situ* analysis and guidance directly addresses the limitations of traditional learning, creating a tight feedback loop between action, observation, and understanding. This application of AI as a live mentor

within a practical cyber range is a primary novel contribution of this work.

C. (L)LMs for Log Analysis and Threat Detection

The technical feasibility of Cyber GYM's core AI feature is supported by a strong and growing body of research into the use of AI, and specifically Large Language Models (LLMs), for cybersecurity operations. AI has proven highly effective in automating routine tasks like log analysis and vulnerability scanning, allowing human analysts to focus on higher-level activities. AI-powered systems can detect threats in real-time by analyzing vast amounts of data to identify patterns, anomalies, and indicators of compromise.

Recent research has specifically explored the use of LLMs for log analysis, demonstrating their ability to recognize patterns, detect anomalies, and pinpoint root causes with greater precision than manual inspection. While some studies note challenges in prompt engineering and potential for high false-positive rates, the field is advancing rapidly, with models being developed to automate correlation across events and classify them with high confidence

PROPOSED METHODOLOGY: THE CYBER GYM PLATFORM

The Cyber GYM platform is designed as a multi-tier, distributed system that provides a seamless, interactive, and secure environment for cybersecurity education. The methodology is centered on a separation of concerns, with distinct functional layers handling the user interface, real-time communication, backend logic, secure command execution, and AI-driven analysis. This section details the conceptual design of these components.

I. Frontend Layer

The user experience is fully delivered through a browser-based frontend, requiring no local installations beyond a standard web browser. Built with HTML, TailwindCSS, and JavaScript, the interface integrates multiple components, including an xterm.js-powered terminal that provides a realistic Linux command-line environment by capturing user input and streaming backend output in real time. A Chart.js-based monitoring dashboard displays live system metrics such as CPU, memory, disk, and network usage, enabling users to immediately observe the effects of their actions. The AI Security Assistant Panel further enhances the experience by presenting real-time explanations, threat classifications, and defensive guidance generated by the AI analysis layer. Secure authentication and session management ensure that only authorized users can access their respective cyber range environments,

maintaining both safety and integrity throughout the platform.

II. Backend Layer

The backend functions as the central nervous system of the platform, orchestrating all data flows and services. Developed in Python using the FastAPI framework and powered by Uvicorn for high-performance asynchronous ASGI capabilities, it is specifically designed to manage numerous concurrent, long-lived WebSocket connections without blocking. Its responsibilities include user management and authentication, such as handling registration, login, and role-based access control to differentiate between administrators and regular users. It also exposes a set of RESTful API endpoints for standard HTTP interactions, including login requests, activity log retrieval, and virtual machine configuration operations. Additionally, the backend acts as the primary orchestrator that mediates communication between the user through WebSocket channels, the target virtual machines via SSH, and the AI model through secured API calls, ensuring seamless and coordinated system operation.

III. WebSocket Communication Layer

To enable the real-time interactivity required for a live terminal, the platform replaces traditional HTTP polling with a persistent, full-duplex communication channel using WebSockets. Implemented through the native WebSocket capabilities of FastAPI and Starlette, this layer is responsible for instantly streaming stdout and stderr from remote SSH sessions to the user's xterm.js terminal, transmitting user-entered commands from the browser to the backend for execution, and continuously broadcasting JSON-formatted system metrics—such as CPU, RAM, and network usage—to the frontend dashboard for live visualization. Establishing a WebSocket connection requires authentication, ensuring that only authorized users can access the interactive terminal and preventing unauthorized access or session hijacking.

IV. Secure SSH Execution Layer

The backend does not execute user commands directly; instead, it relies on a dedicated SSH Execution Layer for secure remote command handling, implemented using the Paramiko library in Python. When a user sends a command, the backend's WebSocket handler forwards it to this layer, which maintains an active, authenticated SSH session with the user's assigned target virtual machine. The command is executed within this controlled session, and the resulting output is captured and returned to the backend. This design establishes a crucial security boundary: the frontend communicates only with the backend via WebSocket, while the backend alone communicates with the target VM through SSH. As a result, users never gain direct network access to the VM, nor do they see or hold SSH credentials. This proxy-based

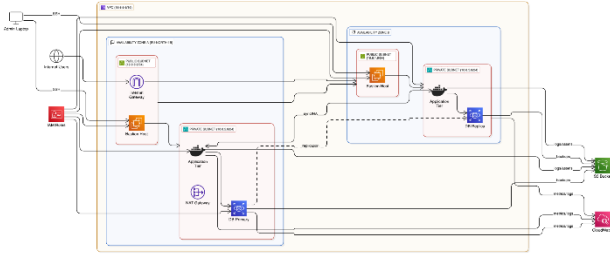
architecture allows the platform to securely manage ephemeral lab environments, prevent unauthorized access, and maintain complete logs of all executed commands for auditing and safety—an essential requirement in an educational cybersecurity setting.

V. AI Threat Analysis Layer

This component serves as the core pedagogical engine of Cyber GYM, integrating with the Google Gemini API to deliver real-time intelligence and contextual support. After a command is executed through the SSH layer, the backend—either automatically or upon user request—sends the command, its output, and relevant command history to the AI layer. This layer formats the information into a carefully designed prompt and submits it to the Gemini API. The AI model processes the input and returns a structured response containing a plain-language explanation of the command, a classification indicating whether the command is suspicious or part of a known attack vector, and defensive recommendations or investigative guidance. This AI-generated analysis is then relayed back to the backend and streamed to the user's AI Security Assistant Panel, providing immediate, expert-level feedback that significantly enhances the learning experience.

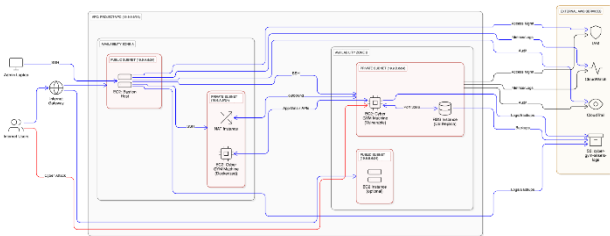
ARCHITECTURE DIAGRAM

The Cyber GYM platform is deployed within a custom AWS Virtual Private Cloud (VPC) in a single region (e.g., eu-north-1) using a 10.0.0.0/16 CIDR block, ensuring strong network isolation and high availability through the use of multiple Availability Zones. A multi-tier subnet architecture separates public and private resources: public subnets in each AZ route internet traffic through an Internet Gateway and host the Bastion Host and the primary EC2 instance running the frontend and backend, while private subnets—without any route to the IGW—contain sensitive components such as the RDS PostgreSQL database and containerized vulnerable target machines. Connectivity is further secured by a NAT Instance in the public subnet, which enables private resources to initiate outbound traffic (e.g., software updates or AI API calls) without exposing them to inbound threats. The compute layer uses EC2 instances for the FastAPI application and for the Bastion Host, while the target environments are deployed as Dockerized EC2 instances isolated in the private subnets. The database layer relies on Amazon RDS with a Multi-AZ PostgreSQL configuration to provide reliable failover, automated backups, and strong protection within the private network.



WORKFLOW

- 1) User logs in through frontend
- 2) Request goes to backend API
- 3) SSH session starts on EC2
- 4) Command execution result is sent back over WebSockets
- 5) Logs are stored in database
- 6) AI module optionally analyzes command history



The Cyber GYM platform workflow begins when a user accesses the system through a web browser using the public IP of the main EC2 instance. After authentication through FastAPI, credentials are verified with the PostgreSQL RDS database. Once logged in, a WebSocket connection is established for real-time communication. Commands entered in the web terminal are securely executed on target EC2 instances via SSH, and the output is instantly streamed back to the browser. Simultaneously, system metrics like CPU and RAM usage are displayed in real time using Chart.js. The backend can also send the user's command and output to the AI Threat Analysis Layer via Google Gemini for intelligent feedback, which is shown in the AI Assistant panel. Finally, all activities, outputs, and AI analyses are logged in the database and archived in AWS CloudWatch and S3 for auditing.

Algorithm

Algorithm 1: AI-Assisted Pedagogical Feedback Loop

```
function on_command(user_command, session_history):
    log_output = execute_ssh(user_command)
    stream_to_user_terminal(log_output)
    log_to_database(user_command, log_output)
    async_call(analyze_with_ai, user_command,
log_output, session_history)

function analyze_with_ai(command, output, history):
    prompt = create_pedagogical_prompt(command,
output, history)
    try:
        analysis_data = gemini_api.analyze(prompt)
    except APIError:
        stream_to_ai_panel({"error": "AI analysis
unavailable"})
    return
    formatted_analysis = format_response(analysis_data)
    stream_to_ai_panel(formatted_analysis)
```

```
function create_pedagogical_prompt(command, output,
history):
    context = "You are a senior cybersecurity instructor
and SOC analyst. "
    context += "A student has executed a command.
Analyze the command, its output, "
    context += "and recent history. Provide an explanation,
identify any attack type, "
    context += "and suggest a defensive measure. Respond
in JSON."
    prompt_data = {
        "context": context,
        "history": history[-5:],
        "command_executed": command,
        "command_output": output
    }
    return json.dumps(prompt_data)
```

II. ML-Based Botnet Detection Model

As a secondary objective, the Cyber GYM platform is designed to incorporate a machine learning model for the detection of specific, well-defined threats, such as the Mirai botnet. While the project report states this as a goal, the full implementation details are not provided. This section, therefore, proposes a conceptual architecture for this module, based on the platform's existing infrastructure and supported by related research.

The objective of this module is to provide a practical, hands-on demonstration of automated threat detection for the student. As the student performs actions (or runs an automated script) that simulate a Mirai infection, a dedicated module will detect and flag this behavior.

VPC Configuration

Custom VPC was created with the following:

NAT Gateway/Instance: Allows private subnet instances to access the internet securely

This is a screenshot of the Amazon Web Services (AWS) console showing the VPC dashboard, which details the configuration and resource map of a specific virtual private cloud.



Fig 2.a



Port 8000/5000 for the application

EC2 instance details (instance ID, type, This image shows the AWS EC2 instance launch process, specifically the pop-up window for **creating a new key pair** named cyber-key using the **RSA** type.IP)



Fig 4.a

Successful database connection from EC2 terminal

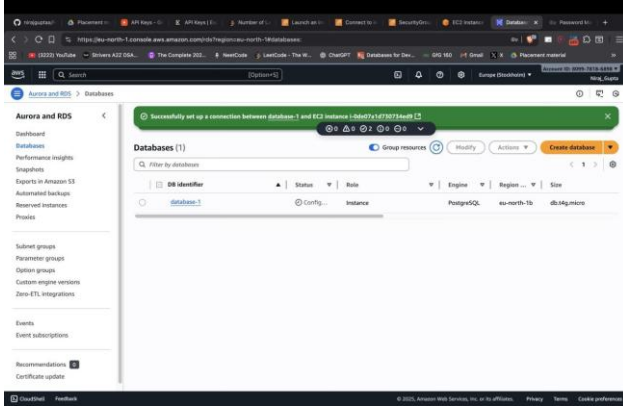


Fig 5.a

Monitoring and Logging (CloudWatch)

Amazon CloudWatch Agent was installed on EC2.

Configured to track:

- CPU Utilization
- Disk Space
- Network In/Out
- Application logs

A CloudWatch log group named **cybergym-app-logs** was created.

Real-time logs and performance metrics are visible from AWS Console.

Alarm: Trigger alert when CPU > 70%

This image shows the AWS CloudWatch console where the user is in the process of selecting a metric to create an alarm. The specific metric selected is NetworkPacketsOut for the EC2 instance labeled CyberGym.

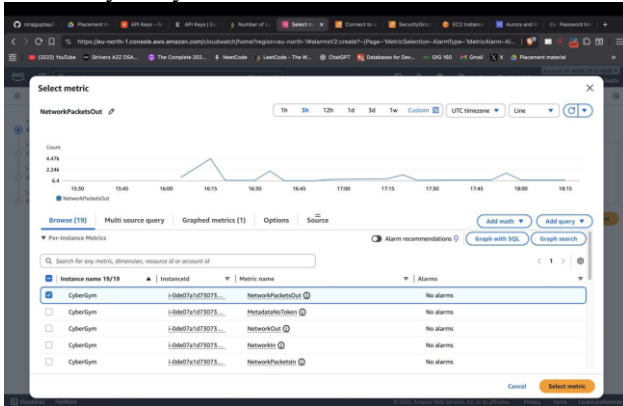


Fig 6.a

This screenshot from AWS CloudWatch displays the dashboard for an alarm named **High-CPU-Usage-CyberGym**, which is currently in the **In alarm** state. Although the alarm is named for CPU usage, the displayed graph shows the **NetworkPacketsOut** metric, indicating that the network traffic has recently exceeded

the configured threshold.

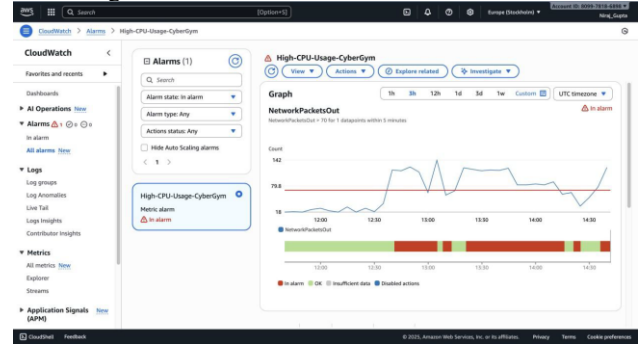


Fig 6.b

Alarm Notification on Registered Main:

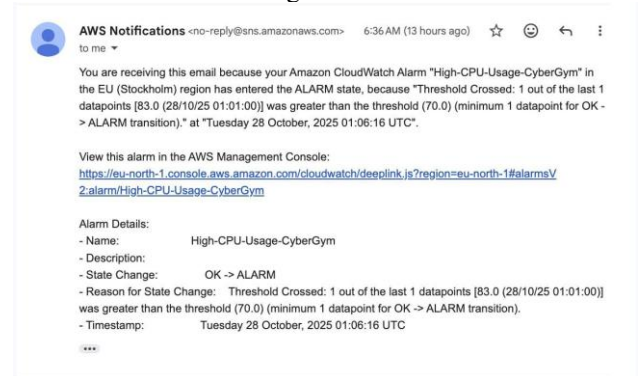


Fig 6.c

Implementation, Results, and Discussion

The conceptual methodology and architecture were successfully implemented and deployed, resulting in a fully functional platform. This section details the specific technologies used, presents the results of the functional validation, and provides a discussion on the platform's security, performance, and cost-optimization.

I. Technology Stack and AWS Service Configuration

The selection of technologies was driven by the requirements for real-time performance, security, scalability, and cost-effectiveness. The core technology stack for the application is summarized in Table I

Table I: TECH STACK

Category	Technology	Purpose
Frontend	HTML, TailwindCSS, JavaScript	User interface, styling, and client-side logic.
	xterm.js	Renders a fully interactive, browser-based terminal.
	Chart.js	Visualizes real-time system metrics (CPU, RAM).
Backend	Python (FastAPI, Uvicorn)	High-performance, asynchronous backend API server.
Real-Time	WebSocket (FastAPI/Starlette)	Full-duplex communication for terminal/metric streaming.
Execution	SSH (Paramiko)	Securely executes user commands on remote VMs.
AI Analysis	Google Gemini API	Provides real-time AI-based threat analysis.
Database	PostgreSQL (AWS RDS)	Data persistence for users, logs, and reports.
Platform	Amazon Web Services (AWS)	Cloud infrastructure

Category	Technology	Purpose
		for all services.
Security	Fernet Encryption, dotenv (.env)	Securely encrypts credentials and manages env variables.

Table II: AWS Service Configuration Summary

AWS Service	Configuration Detail	Purpose and Justification
VPC	Custom VPC (10.0.0.0/16), 2 Public Subnets, 2 Private Subnets across 2 AZs.	Network isolation. Private subnets secure the database and target VMs; public subnets host web-facing resources.
Internet Gateway	1x IGW attached to VPC and Public Route Table.	Provides internet access for resources in the public subnets (e.g., the web application EC2).
NAT Instance	1x t2.micro EC2 instance, referenced in Private Route Table.	Allows instances in private subnets (e.g., RDS, target VMs) to initiate

<i>AWS Service</i>	<i>Configuration Detail</i>	<i>Purpose and Justification</i>
		outbound connections (for updates or API calls) without being publicly exposed.
EC2	1x t2.micro (Ubuntu) for Web App/Backend.	Hosts the FastAPI application. t2.micro is used to align with the AWS Free Tier, minimizing cost for educational use.
Security Groups	Web App SG: Inbound 8000/443 (All), 22 (Bastion). RDS SG: Inbound 5432 (Web App SG).	Enforces a least-privilege firewall. The database is only accessible from the application, not the internet.
RDS	1x db.t3.micro PostgreSQL (Free Tier), 20GB SSD, in Private Subnet.	Provides a secure, managed, and auto-backed-up database for application data. Private subnet deployment

<i>AWS Service</i>	<i>Configuration Detail</i>	<i>Purpose and Justification</i>
		It is a critical security feature.
S3	Bucket: cybergym-app-logs, Block Public Access (On).	Durable, low-cost storage for application logs, archives, and RDS backups. Access is restricted via IAM Roles.
CloudWatch	Agent installed on EC2. Log Group: cybergym-app-logs. Alarm: CPU Utilization > 70%.	Provides comprehensive monitoring, logging, and automated alerting for the platform's health and performance.
IAM	1x IAM Role (e.g., EC2CloudWatchAgentRole) attached to EC2 instance.	Securely grants the EC2 instance permissions to access other AWS services (CloudWatch, S3) without using static, hardcoded credentials.

System Deployment and Functional Validation:

The Cyber GYM platform was successfully developed and deployed on AWS, accessible via its public IP. Functional validation confirmed that all system components operated as intended, providing a smooth and interactive experience. The xterm.js terminal delivered near-native SSH responsiveness with real-time CPU, memory, and storage metrics displayed dynamically. The AI-driven analysis panel accurately identified and explained simulated attack scenarios like port scans and brute-force attempts. AWS integration with EC2, RDS, and CloudWatch proved reliable under multiple concurrent sessions. Overall, the platform met all objectives for virtualization, secure access, real-time monitoring, and AI-assisted cybersecurity learning.

Security and Performance Analysis:

Cyber GYM was built with a multi-layered, security-first architecture. Application-level security ensures that credentials and SSH keys are encrypted and securely stored, while the network design isolates critical components within private subnets protected by strict security groups. IAM roles and authenticated WebSocket sessions further enhance cloud and session security. Performance optimization was achieved through WebSockets and FastAPI's asynchronous handling, enabling low-latency, real-time responses even with many users. Cost efficiency was maintained by leveraging AWS Free Tier resources, limiting log retention, and triggering AI analysis only on demand. This balance of strong security, real-time performance, and cost optimization makes the platform both educationally valuable and operationally sustainable.

Conclusion

The Cyber GYM project successfully delivers a practical, interactive, and AI-assisted cybersecurity training environment that bridges the gap between theoretical learning and real-world defensive skills. By combining a real-time browser-based terminal, secure SSH-based command execution, and a multi-tier AWS cloud architecture, the platform provides a scalable and safe sandbox for hands-on experimentation while remaining cost-effective through Free Tier-optimized design. Its most novel contribution is the integration of Google Gemini as a real-time pedagogical assistant, offering immediate, context-aware explanations and defensive guidance that transform simulations into guided learning experiences. Functional validation shows stable performance, accurate AI insights, and strong security

across all layers. Overall, Cyber GYM stands as a complete and impactful system that advances modern cybersecurity education.

Future Work

While the current Cyber GYM platform offers a strong foundation, its modular architecture enables substantial future enhancements. Planned developments include support for advanced multi-user cybersecurity scenarios, such as full Security Operations Center (SOC) simulations with coordinated Red Team and Blue Team exercises. The platform can also be improved through scenario automation using tools like Ansible or Terraform, enabling instructors to deploy complete vulnerable lab environments with a single action. AI and ML capabilities represent another major area for expansion, including the development of domain-specific generative models trained on cybersecurity datasets and a more comprehensive library of malware and botnet detectors. Additionally, future work will focus on formal integration into academic curricula by creating structured learning modules, lab activities, and assessment frameworks that align with university-level cybersecurity education.

References

G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. on Operating Systems Principles (SOSP '07)*, Stevenson, WA, USA, Oct. 2007, pp. 205–220.

I. Fette and A. Melnikov, "The WebSocket Protocol," *RFC 6455*, Dec. 2011. [Online]. Available: <https://doi.org/10.17487/RFC6455>.

Intelligent Detection of IoT Botnets Using Machine Learning and Deep Learning - MDPI, accessed November 12, 2025, <https://www.mdpi.com/2076-3417/10/19/7009>

Leveraging Machine Learning for Botnet Attack Detection in Edge-Computing Assisted IoT Networks - arXiv, accessed November 12, 2025, <https://arxiv.org/html/2508.01542v1>

"KYPO Cyber Range Platform," CONCORDIA H2020. [Online]. Available: <https://crp.kypo.muni.cz/>.

Fortinet, "Artificial Intelligence (AI) in Cybersecurity," 2024. [Online]. Available: <https://www.fortinet.com/resources/cybergl ossary/artificial-intelligence-in-cybersecurity>.
S. R. Ahmad, A. S. Al-Shboul, and M. A. F. Al-Husainy, "Innovating Cybersecurity Education Through AI-augmented Teaching," *ResearchGate*, June 2024. [Online]. Available: <https://www.researchgate.net/publication/381651850>.

The Emerging Threat of Ai-driven Cyber Attacks: A Review - Taylor & Francis Online, accessed November 12, 2025, <https://www.tandfonline.com/doi/full/10.1080/08839514.2022.2037254>

J. Chen, "Model Algorithm Research based on Python Fast API," *Frontiers in Science and Engineering*, vol. 3, no. 9, pp. 7–10, 2023.
J. Forcier, "Paramiko: A Python implementation of SSHv2," 2025. [Online]. Available: <https://www.paramiko.org/>.

T. Tiangolo, "FastAPI: A modern, fast (high-performance) web framework," 2024. [Online]. Available: <https://fastapi.tiangolo.com/>.

Gemini Team *et al.*, "Gemini: A family of highly capable multimodal models," *arXiv:2312.11805*, Dec. 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2312.11805>.

Artificial Intelligence-Based Cyber Security in the Context of Industry 4.0—A Survey - MDPI, accessed November 12, 2025, <https://www.mdpi.com/2079-9292/12/8/1920>

Keywords

Cyber Range; Cybersecurity Education; Cloud Computing; Amazon Web Services (AWS); AI-Assisted Learning; Log Analysis; Large Language Models (LLM); WebSocket; Secure Simulation.